

LLM-Assisted Security Audit and Fuzzing

Nim `std/asynhttpserver`

COMP431 / Spring 2026 / Local Defensive Audit

This report documents a local educational reproduction of an already-fixed upstream Nim bug. No remote system was tested, and the result should be read as evidence for the project methodology, not as a claim about an active deployment risk.

Project Overview

This project is a local security-audit prototype for Nim's standard-library HTTP server module, `std/asynchttpserver`. The goal was to compare two vulnerability-discovery techniques:

1. Coverage-guided fuzzing with libFuzzer and sanitizers.
2. AI-assisted static review using an LLM prompt and manual validation.

The project follows the “LLM-assisted Security Audit” course topic. It implements a working local auditing setup, runs simulated malformed inputs against parser code, records evidence, and evaluates the usefulness of each technique. No remote system was tested.

The main result was a confirmed denial-of-service crash in the HTTP protocol parser. A malformed request-line protocol token, `HTTP/`, can raise an uncaught `IndexDefect` in the older version tested for this project. The Nim project has since fixed this upstream in PR 25793, which validates the finding as a real previously existing bug. The older Nim version was used intentionally to study discovery and validation techniques.

Target and Background

The audited file was:

```
Nim/lib/pure/asynhttpserver.nim
```

This module implements an asynchronous HTTP server. It is not intended to be a production web server directly exposed to the Internet, but applications may use it locally or behind another service. HTTP request data is attacker-controlled in the general model, so request parsing remains a meaningful security boundary for defensive testing.

The most important parsing path is `processRequest`, which reads:

- the request line,
- headers,
- optional body data from `Content-Length`,
- optional chunked body data,
- connection-management headers.

The helper most relevant to the confirmed crash is `parseProtocol`, which parses strings such as `HTTP/1.1` into a tuple containing the original string, major version, and minor version.

Threat Model and Risk Analysis

For the experiment, the attacker model is an unauthenticated client who can send malformed HTTP data to an application using the older vulnerable parser. The attacker controls bytes in the request line, headers, and body.

Risk category	Why it matters in parser testing
Process crash	One malformed request can terminate a process in the vulnerable version.
Memory exhaustion	Large or repeated inputs can force allocation pressure.
Connection exhaustion	Slow reads without timeouts can hold sockets open.
HTTP logic errors	Incorrect header or body handling can confuse applications using the server.

This project did not attempt remote code execution or exploitation of any third-party deployment. The evaluation was a local defensive audit.

Prototype Design

The prototype has four parts:

Component	Purpose	Location
Fuzz harness	Exercises the full request parser with generated input	experiments/fuzzing/harness/
Seed corpus	Small benign starting inputs for libFuzzer	experiments/fuzzing/corpus/
Saved evidence	Crash input	experiments/fuzzing/results/
AI review notes	Prompt, raw model outputs, extracted findings	ai_findings/

The fuzz harness does not start a real network server. Instead, it directly calls the full request parsing logic that would normally receive data from a client. The harness calls `parseFullRequestOriginal`, which exercises all parsing stages in sequence: method, URI, protocol, headers, and body. This makes testing faster and deterministic while still exercising code that handles untrusted HTTP strings.

The harness also supports a **standalone replay mode** (`-d:fuzzStandalone`) that reads crash inputs from command-line file arguments and replays them through the same parsing code — no libFuzzer, no coverage tracking, no mutation.

Security Techniques Used

Coverage-Guided Fuzzing

The first technique is fuzz testing. libFuzzer mutates inputs and uses coverage feedback to explore new program paths. The harness was compiled with:

- Clang,
- libFuzzer,
- AddressSanitizer,
- UndefinedBehaviorSanitizer.

This is a local attack simulation because it repeatedly sends malformed inputs to code that normally processes untrusted HTTP data.

AI-Assisted Static Review

The second technique is LLM-assisted review. A single structured prompt asked models to inspect `asynhttpserver.nim` and report findings with:

- title,
- vulnerability type,
- location,
- triggering input,
- root cause,
- exploitability,
- classification.

DeepSeek V4 Pro, GLM-5.1, and Kimi K2.6 produced usable transcripts. Their output was treated as hypotheses. Each claim was checked against source code and fuzzing evidence before classification.

Experimental Methodology

Building the Fuzzer

The build command lives in `scripts/build_fuzzers.sh`. The core compilation command is:

```
nim c \
  --cc:clang \
  -d:noSignalHandler \
  -d:useMalloc \
  --noMain:on \
  --passC:-fsanitize=fuzzer,address,undefined \
  --passC:-g \
  --passL:-fsanitize=fuzzer,address,undefined \
  --passL:-g \
  experiments/fuzzing/harness/asynhttpserver_fuzzer.nim
```

The important flags:

- `--cc:clang`: libFuzzer requires Clang.
- `-d:noSignalHandler`: lets sanitizers report crashes directly, not Nim's own handlers.
- `-d:useMalloc`: routes allocation through the system allocator so ASan can instrument it.
- `--noMain:on`: lets libFuzzer provide `main()`.
- `--passC/--passL -fsanitize=fuzzer,address,undefined`: enables libFuzzer, ASan, and UBSan.

The build produces `experiments/fuzzing/harness/asynhttpserver_fuzzer`.

Seed Corpus

The seed corpus is a set of small valid inputs that exercise target paths before mutation. The project uses three seed files:

- `seed_get_http11`: a minimal valid GET request.
- `seed_post_content_length`: a POST request with Content-Length.
- `seed_post_chunked`: a POST request with chunked transfer encoding.

The seeds are intentionally well-formed. The fuzzer mutates them into malformed inputs and observes whether new paths or crashes appear.

Running the Fuzzer

The run command lives in `scripts/run_fuzzing.sh`. It builds the fuzzer, copies the seed corpus into a temporary working directory, and invokes:

```
ASAN_OPTIONS=detect_leaks=0 \
  experiments/fuzzing/harness/asynhttpserver_fuzzer \
  -print_final_stats=1 \
  -max_total_time=60 \
  -artifact_prefix="$WORKDIR/results/" \
  "$WORKDIR/corpus"
```

The important pieces:

- `ASAN_OPTIONS=detect_leaks=0`: disables leak detection to avoid noise in a fuzz target.
- `-max_total_time=60`: bounds the run to 60 seconds.
- `-artifact_prefix`: where to save crash inputs.

Interpreting Fuzzer Output

A typical run that found the crash produced:

```
#2373 REDUCE cov: 3798 ft: 8650 corp: 106/4434b lim: 67 exec/s: 0 rss: 58Mb
==67332== ERROR: libFuzzer: fuzz target exited
SUMMARY: libFuzzer: fuzz target exited
artifact_prefix='/tmp/tmp.XXXXXX/results/'; Test unit written to .../crash-
bc01f5feb2d0b6b6e46410cfcc0082629a815e22
Base64: R0VUIC0gSFRUUC8gSFRUUC8xLjEgSA==
stat::number_of_executed_units: 2847
```

Key fields:

- `#2373`: the fuzzer had executed 2,373 test cases.
- `REDUCE`: libFuzzer was minimizing the crashing input.
- `cov: 3798`: 3,798 unique code edges were reached.
- `SUMMARY: libFuzzer: fuzz target exited`: the target terminated unexpectedly.
- `Base64: R0VUIC0gSFRUUC8gSFRUUC8xLjEgSA==`: decodes to `GET - HTTP/ HTTP/1.1 H`, a request whose protocol field is `HTTP/`.

Crash Artifact

The saved crash file `experiments/fuzzing/results/F-001_crash_input` is the minimized 10-byte input `PUT HTTP/`. When split by spaces, the third field is `HTTP/` — a protocol string with no version digits at all. The `parseProtocolOriginal` function accepts the `HTTP/` prefix, then unconditionally increments `i` to skip the dot separator. Since the string is only 5 characters, `i` moves past the end, and the subsequent call to `parseSaturatedNatural` triggers `IndexDefect`.

Verifying with Standalone Replay

A fuzzer artifact is evidence, but it should be verified independently. The harness supports a standalone replay mode that feeds saved inputs through the same parsing code without the fuzzer engine:

```
nim c -d:fuzzStandalone --panics:on --mm:arc \
  experiments/fuzzing/harness/asynhttpserver_fuzzer.nim

./experiments/fuzzing/harness/asynhttpserver_fuzzer \
  experiments/fuzzing/results/F-001_crash_input
```

Output:

```
StandaloneFuzzTarget: running 1 inputs
Error: unhandled exception: index out of bounds: 7..5 notin 0..5 [IndexDefect]
```

The stack trace confirms the call chain: `parseFullRequestOriginal` → `parseProtocolOriginal` → `parseSaturatedNatural` → crash. For comparison, replaying the well-formed seed corpus produces exit code 0.

AI Review Workflow

The AI workflow was separate from fuzzing:

1. A structured prompt in `ai_findings/VULN_DISCOVERY_PROMPT.txt` described the target, required output fields, and attack surfaces.
2. The same prompt and source were submitted to DeepSeek V4 Pro, GLM-5.1, and Kimi K2.6.
3. Raw outputs were saved in `ai_findings/<model>/output.txt`.
4. Findings were extracted into `ai_findings/<model>/findings_extracted.md`.
5. Each finding was validated against source code and fuzzing evidence.

The critical rule was that AI output was treated as a list of hypotheses, not confirmed facts.

Distinguishing Bugs from False Positives

The validation checklist was:

1. Can we replay the crash independently?
2. Can we trace the code path from input to failure?
3. Is the AI misreading runtime behavior?
4. Is the claim bounded by existing limits?
5. Does the fuzzer agree?

The HTTP/ finding passed the strongest test: standalone replay triggers the same failure as the fuzzer artifact. DeepSeek and GLM noticed the suspicious HTTP/ case, but they incorrectly predicted it would be accepted as version 0.0. Runtime evidence showed the opposite.

Confirmed Vulnerability

F-001: parseProtocol IndexDefect

Relevant source from the vulnerable version:

```
proc parseProtocol(protocol: string): tuple[orig: string, major, minor: int] =
  result = default(tuple[orig: string, major, minor: int])
  var i = protocol.skipIgnoreCase("HTTP/")
  if i != 5:
    raise newException(ValueError, "Invalid request protocol. Got: " & protocol)
  result.orig = protocol
  i.inc protocol.parseSaturatedNatural(result.major, i)
  i.inc
  i.inc protocol.parseSaturatedNatural(result.minor, i)
```

Expected behavior:

- HTTP/1.1 should parse successfully.
- Incomplete version values such as HTTP/ should not crash the process.
- A handled ValueError would be acceptable for malformed input.

Actual behavior:

- HTTP/ passes the prefix check.
- i becomes 5.
- The string length is also 5.
- parseSaturatedNatural consumes zero digits and leaves i at 5.
- The old code executes i.inc unconditionally.
- i becomes 6, past the end of the string.
- The second numeric parse starts out of bounds.
- Nim raises IndexDefect.
- processRequest catches only ValueError, so the IndexDefect escapes.

Why It Is Real

1. Standalone replay confirms the crash with a full stack trace.
2. The root cause is one clear unconditional i.inc.
3. The fuzzer and replay agree on the same defect.
4. The upstream Nim project merged PR 25793 with the same guard.
5. The error message precisely identifies the failure: index out of bounds: 6..4 not in 0..4.

Fuzzing Results

The fuzzing campaign found the crash after approximately 2,847 executions with 3,798 code edges covered. The original crash artifact was 34 bytes; libFuzzer's `-minimize_crash=1` reduced it to the 10-byte input `PUT HTTP/` — a request line whose protocol field is exactly `HTTP/`.

Campaign	Harness	Result	Executions	Coverage
1	Original parser	Crash found	2,847	3,798

Replay

The replay command in `scripts/replay_crash.sh` passes the saved artifact back to the fuzzer in single-run mode (`-runs=1`) for verification. The standalone replay mode (above) serves the same purpose without the fuzzer engine.

AI-Assisted Review Results

Three models produced usable transcripts: DeepSeek V4 Pro, GLM-5.1, and Kimi K2.6.

DeepSeek V4 Pro

Result	Count
Confirmed	0
Likely	6
Unlikely	1
False positive	1

DeepSeek’s useful likely findings included chunked body growth without `maxBody`, missing receive timeouts, header-limit behavior, connection-header first-value logic, POST-only chunked handling, and case-sensitive chunked comparison. Its important false positive was the claim that `HTTP/` would be accepted as version `0.0`.

GLM-5.1

Result	Count
Confirmed	0
Likely	5
Unlikely	1
False positive / withdrawn	4

GLM overlapped with DeepSeek on chunked body growth and missing receive timeouts. Like DeepSeek, it reached the wrong runtime conclusion about `HTTP/`, so it is not counted as finding the confirmed crash.

Kimi K2.6

Kimi produced five candidate findings, all downgraded to likely because no dynamic reproducers were built. It did not report the confirmed `HTTP/` fuzzing crash.

Overall AI Summary

Model	Usable tran-script	Reported	Confirmed	Likely	False / with-drawn
DeepSeek V4 Pro	Yes	8	0	6	1
GLM-5.1	Yes	10	0	5	4
Kimi K2.6	Yes	5	0	5	0

The AI outputs were useful for review coverage, but none of the usable transcripts correctly identified the bug that fuzzing confirmed.

Evaluation

Fuzzing was better at producing confirmed evidence. It gave a minimized input, a crash log, and a way to replay the result.

AI review was useful in a different role. It found broader design concerns that a small parser fuzzer does not fully model — timeout behavior, chunked body growth, header storage before limit checks. The AI outputs still required manual checking because none correctly identified the confirmed crash.

Criterion	Fuzzing	AI-assisted review
Confirmed bugs	1	0
Main strength	Reproducible runtime evidence	Broad static hypothesis generation
Main weakness	Limited to modeled parser paths	Can misread runtime behavior
Evidence quality	Crash input, stack trace, replay	Source-level reasoning requiring validation

AI output should be used as a review checklist. Each item needs either a reproducer, a complete static trace, or a clear reason why it is not exploitable.

Remediation Discussion

The fix does not need many new error branches. The crash is caused by one unconditional index increment. The minimal fix is:

```
result = default(tuple[orig: string, major, minor: int])
var i = protocol.skipIgnoreCase("HTTP/")
if i != 5:
    raise newException(ValueError, "Invalid request protocol. Got: " & protocol)

result.orig = protocol
i.inc protocol.parseSaturatedNatural(result.major, i)
if i < protocol.len:
    inc i # Skip .
i.inc protocol.parseSaturatedNatural(result.minor, i)
```

This only prevents the parser from manufacturing an out-of-bounds index when the version is incomplete.

Upstream Fix Confirmation

The Nim project merged this exact fix in [PR 25793](#). The upstream change is:

- Old: `i.inc # Skip .`
- New: `if i < protocol.len: inc i # Skip .`

The upstream fix validates the finding and the root-cause analysis.

Limitations

The fuzzer does not run a complete server. It tests parser paths but does not model:

- real socket timing,
- slow clients,
- many simultaneous connections,
- callback behavior,
- operating-system file-descriptor limits.

The three AI model outputs were validated manually. The confirmed impact was denial of service. No evidence of memory corruption or remote code execution was found.

Lessons Learned

Reproducible evidence matters. A crash claim is strongest when it includes the exact input, the fuzzer log, and an independent replay.

Distinguish tools from evidence. A fuzzer produces direct runtime evidence. An AI model produces static hypotheses. A finding is not confirmed because a model sounds confident — it is confirmed because it can be reproduced.

Minimization is valuable. libFuzzer reduced the crash input to something a human can reason about. The protocol field `HTTP/` is clearly missing its version digits, pointing directly to the unconditional `i.inc`.

An LLM can help organize an audit and point toward interesting code paths, but it can be wrong about runtime behavior. DeepSeek and GLM noticed the suspicious `HTTP/` case but reached the opposite conclusion from the actual program, while Kimi missed that crash entirely.

Conclusion

The project implemented a working local security-audit prototype and used it to reproduce a real, already-fixed crash in Nim's `std/asynhttpserver`. Fuzzing produced the strongest confirmed result: an uncaught `IndexDefect` from `parseProtocol("HTTP/")` in the older version tested. The upstream Nim project has since merged a fix for this exact issue in PR 25793

AI-assisted review produced useful likely findings but no confirmed finding. All usable model outputs required validation, and all over-claimed at least some findings as confirmed. The best workflow was to use AI for broad review and hypothesis generation, then use fuzzing and replay to validate claims.

The useful unit is not “AI found a bug.” The useful unit is a reproducible pipeline that turns a suspicious idea into a tested input and a verified fix.